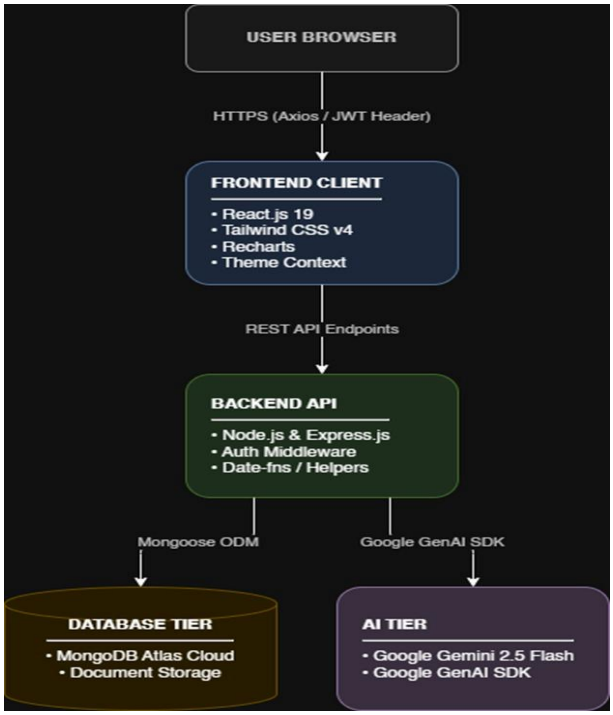
SYSTEM DESIGN SPECIFICATION: AURAHABIT

Course: CSE4204

Project Title: AuraHabit: AI-Powered Smart Habit Formulation & Behavior Insights Tracker

1. System Architecture Diagram

The architecture of AuraHabit follows a decoupled, multi-tier cloud design to guarantee clear separation of concerns, scalability, and seamless AI data streaming.



Architectural Component Breakdown:

Frontend (React.js 19 & Tailwind CSS v4): Handles responsive layout presentation, dark mode client-side theme states, and asynchronous HTTP calls via Axios. It uses **Recharts** for rendering high-fidelity habit performance trends.

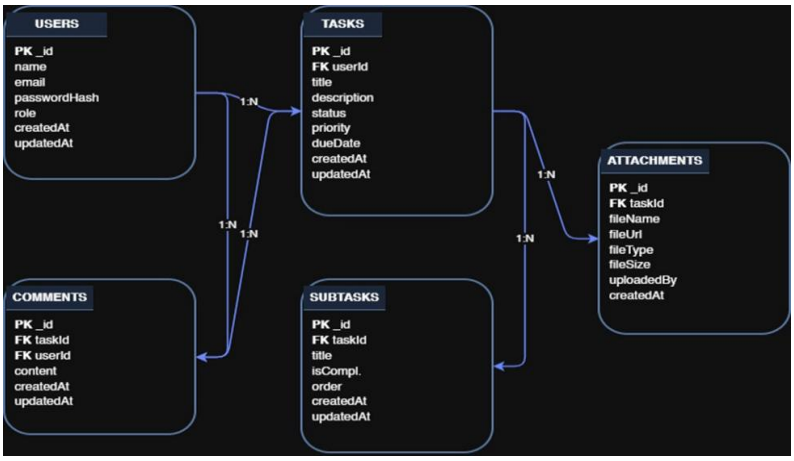
Backend API (Node.js & Express): Serves as the central application router, applying JSON Web Token (JWT) verification middleware to incoming requests and parsing date objects cleanly.

Database (MongoDB Atlas): A cloud-managed NoSQL environment storing data as flexible, schema-validated documents through a **Mongoose ODM** mapping layer.

AI Service (Google Gemini 2.5 Flash): Integrated natively using the **Google GenAI SDK** to evaluate habits, produce suggestions, and deliver contextual text responses.

2. Entity-Relationship (ER) Diagram / Data Model

To support application features—including user ownership, customized habit parameters, subtasks, and structural comments—the collection data layers relate dynamically as modeled below:

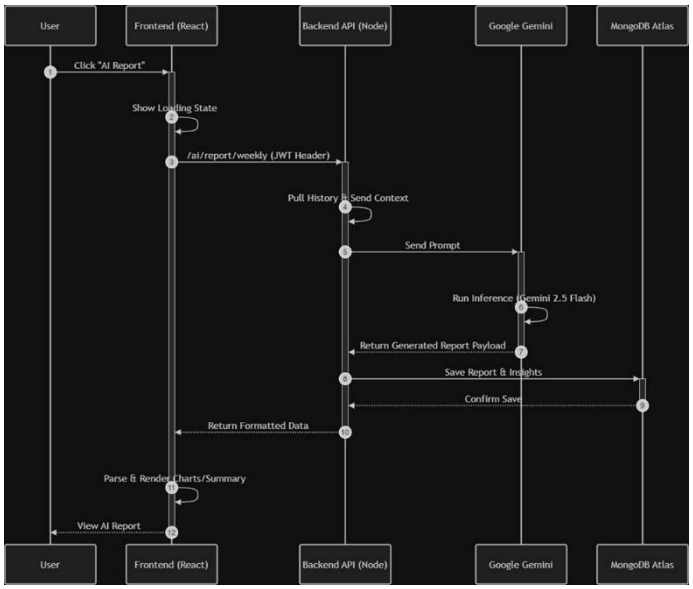


Structural Integrity Constraints:

1. **Users to Tasks:** A single User document maps to zero or many items in the Tasks collection via the indexed userId key field (1→∞)
2. **Tasks to Subtasks:** Tasks can be broken down granularly into structured sub-steps ordered through numerical index priorities (1→∞)
3. **Comments & Attachments:** Both reference an originating parent context ID (taskId) to store supporting files, upload sizes, or collaborative conversation histories securely within dedicated tables.

3. Updated Use Case Diagram

The updated diagram captures all major core capabilities alongside the specialized, direct downstream sub-dependencies ≪ include ≫ assigned directly to our cloud AI model.



4. Activity Diagram (Workflow of a Major Feature)

Feature: Generate AI Weekly Report (UC-5)

The detailed transactional timeline spanning our four operational layers executes sequentially through the layout below:

5. API Design Document

Authentication APIs

POST /api/auth/register

1. **Purpose:** Registers a new user account profile securely in the cloud cluster database.
2. **Input Data:** {"name": "string", "email": "string", "passwordHash": "string"}
3. **Expected Output:** {"success": true, "message": "User registered successfully", "userId": "65b2f1..."}

POST /api/auth/login

1. **Purpose:** Authenticates account credentials and yields a stateless bearer token authorization string.
2. **Input Data:** {"email": "string", "passwordHash": "string"}
3. **Expected Output:** {"success": true, "token": "eyJhbGciOi...", "user": {"id": "65b2f1...", "name": "Syed Mostofa Rafid"}}

Habit & Task Tracking APIs

POST /api/tasks

1. **Purpose:** Creates an explicit trackable activity row bound directly to the authenticated caller profile.
2. **Input Data:** {"title": "string", "description": "string", "priority": "High", "dueDate": "2026-06-20"}
3. **Expected Output:** {"success": true, "taskId": "65b302...", "message": "Task cataloged successfully"}

GET /api/tasks

1. **Purpose:** Pulls an array containing all historical task objects and state variables matching the authenticated token's unique ID.
2. **Input Data:** None (Requires header validation token: Authorization: Bearer <token>).
3. **Expected Output:** {"success": true, "results": [{"_id": "65b302...", "title": "Run 5k", "status": "Pending"}]}

AI Integrated Insights APIs

POST /api/ai/report/weekly

1. **Purpose:** Triggers context parsing of user histories to construct analytical performance forecasts via Gemini inference patterns.
2. **Input Data:** {"currentDate": "2026-06-15"}

3. **Expected Output:** {"success": true, "insightsSummary": "Your target habits peaked during morning blocks, falling roughly 40% on Thursdays...", "chartsData": {"completionRate": 82}}

POST /api/ai/coach/chat

1. **Purpose:** Passes an active chat message through to the dynamic behavioral assistant instance.
2. **Input Data:** {"messageText": "I fell out of line with my habits for three days straight. Help me get back on track."}
3. **Expected Output:** {"success": true, "replyText": "Lapses are normal parts of behavioral learning patterns. Let's apply a 5-minute scaffolding ruleset today to reduce cognitive resistance."}

6. AI Integration Workflow

• Why AI is Needed

Standard habit apps act as passive, forgettable checkboxes. They place the entire motivational and cognitive burden onto the user. AuraHabit leverages artificial intelligence to transform the tracking tool from a static list into an **active behavioral partner** capable of forecasting failure cycles, streamlining goal scaling, and removing friction before user engagement falls off.

• Which AI Service/Model Will Be Used

The production application utilizes the **Google Gemini 2.5 Flash Model** connected securely via the official **Google GenAI SDK**. This specific model provides the low latency needed for fast, reactive UI interactions, while maintaining strong schema parsing capabilities for structuring analytical reports.

• Input to the AI System

The engine takes an explicit developer prompt injected with live database context variables, including:

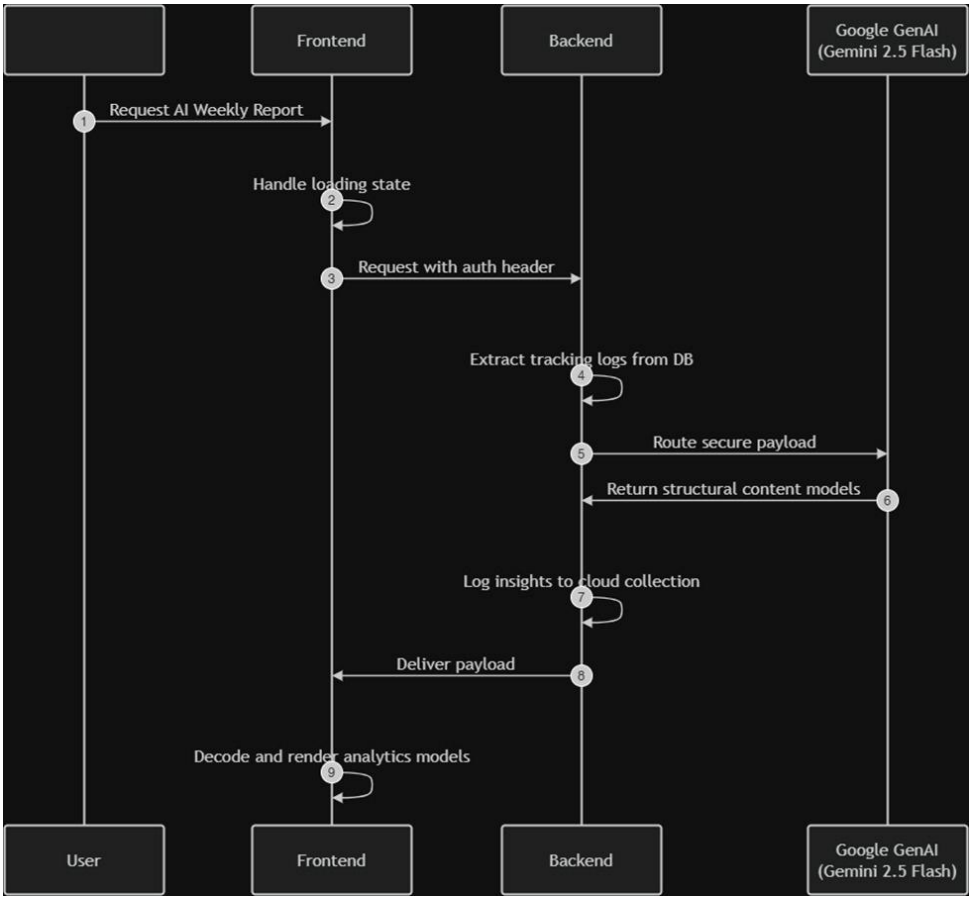
1. A historical log array containing past tracking success percentages over the previous 30 days.
2. Self-reported contextual inputs (e.g., categorical priority types or custom overdue status parameters).

• **Output from the AI System**

1. **Conversational Text Blocks:** Tailored, encouraging guidance on behavioral psychology directly inside the interactive chat module.
2. **Structured JSON Models:** Categorized weekly reports, actionable optimization patterns, and clear insights delivered cleanly to the client data stream.

• **How AI Improves the Project**

By adding data-driven, predictive tracking capabilities, the application shifts away from an ordinary checklist model. It builds deep, custom value for user retention, creating a competitive, stand-out submission for our university project showcase.



7. Conclusion & Project Readiness Summary

The architectural and system design frameworks established across this document provide a comprehensive, production-ready blueprint for **AuraHabit: AI-Powered Smart Habit**

Formulation & Behavior Insights Tracker. By cross-referencing our core operational components, the team has successfully achieved the following structural alignments:

Architectural Alignment: The system design establishes a clear, decoupled interaction layer where a highly responsive **React.js 19** frontend connects seamlessly to an asynchronous **Node.js & Express** backend. Data persistence is fully handled by a scalable cloud cluster hosted on **MongoDB Atlas**, while intensive behavioral computations are delegated natively to the **Google Gemini 2.5 Flash Model** using the official SDK.

Data Model Integrity: The structured **Entity-Relationship (ER) Diagram** ensures that user profiles map dynamically to operational trackers, supporting cascading subtasks, relational commenting systems, and storage metadata with strict primary and foreign key constraints.

Workflow Validation: The **Use Case Diagram** maps out exactly 12 core capabilities that give an authenticated user full control over their tracking data. This is directly complemented by the **Activity Diagram for UC-5 (Generate AI Weekly Report)**, which traces a rigorous data flow verifying exactly how user tokens move from frontend button clicks through backend validation, database state lookups, external AI inference cycles, and back to user interface dashboards.

API Security & Separation of Concerns: Core REST API endpoints separate critical application pathways—including secure JWT-driven authorization flows, CRUD actions for standard tasks, and data streams running into the AI engine layer.

With all major workflows mapped, schemas defined, and API data expectations explicitly structured, the system design phase is successfully complete. This blueprint ensures structural consistency, prevents future technical debt, and allows the development team to confidently transition into writing code, setting up repository infrastructure, and implementing features for the upcoming university showcase.

Submitted By:

- Syed Mostofa Rafid
  CSE- 11220320892
- Huzaifa Islam Taseen
  CSE- 11220320883
- Shekh Sirajum Munir
  CSE- 11220320894
- Hemandra Nath Mondal
  CSE - 11220320873